



S.G. Ganesh

Types of Bugs

In this column, we'll look at four types of bugs, named after popular scientists. The classification is interesting—we'll understand how strange bugs can be!

*J*im Gray, in his popular paper (see *References*) originally proposed the classification of bugs as *Bohrbugs* and *Heisenbugs*, named after well-known scientists. Today, there are more bug types known to us, so we'll also look at two other categories of them.

Bohrbugs: Most of the bugs that we come across are reproducible, and are known as *Bohrbugs*. These are named after Niels Bohr, who proposed a simple and easy-to-understand atomic model in 1913. In Bohr's model, things like the path and momentum of an electron in an atom are predictable. Similarly, *Bohrbugs* are predictable—you can reproduce them if you run the software with similar conditions. For example, when the program crashes with a null-pointer access, it always crashes there for a given input; so you can easily reproduce it.

Heisenbugs: All experienced programmers have faced situations where the bug that crashed the software just disappears when the software is restarted. No matter how much time and effort is spent trying to reproduce the problem, the bug eludes us. Such bugs were named *Heisenbugs*, after Werner Heisenberg, who is known for his 'uncertainty principle'. According to his theory, it is not possible to accurately or certainly determine the position and velocity of an electron in an atom at a particular moment. When bugs change their behaviour when you try to debug, probe or isolate, they are called *Heisenbugs*. It can happen, for example, when you use uninitialised variables. When the program is run, it will access variables that are uninitialised, and hence result in a bug. However, when you try to debug the program, the program might work just fine, because many debuggers initialise uninitialised variables to zeros, and so you might not hit the problem!

Mandelbugs: When the cause of the bug is too complex to understand, and the resulting bug appears chaotic, it is called a *Mandelbug*. These are named after Benoît Mandelbrot, who is considered the father of fractal geometry (fractals are complex, self-similar structures). A bug in an operating system that depends on scheduling is an example of a *Mandelbug*.

Schroedinbug: Sometimes, you look into the code, and find that it has a bug or a problem that should never have allowed it to work in the first place. When you try out the code, the bug promptly shows up, and the software fails! Though it sounds very uncommon, such bugs do occur and are known as *Schroedinbugs*. They are named after the scientist Erwin Schrödinger, who proposed a theoretical 'cat experiment'. In quantum physics, quantum particles like atoms could exist in two or more quantum states, but Schrödinger suggested that in more classical objects like a cat which is made up of many atoms, existing in two states was impossible. He theorised about a scenario in which a cat is kept in a sealed chamber, with a vial of poison (attached to a radioactive atom). If the atom decayed, the vial would be smashed and the poison would leak, killing the cat. But with the chamber sealed, there would be no way to know whether the cat was dead or alive. So till the chamber is opened, theoretically, the cat could be in either of two states—dead or alive. In quantum physics, this is called a 'superposition state', where the cat is both alive and dead!

Coming back to bugs, by merely observing the problem in the code, you change the outcome—either the software works or breaks. So these kinds of bugs are known as *Schroedinbugs*.

There are other types of bugs that don't come under these categories. For instance, 'aging-related bugs' occur only after the software runs for a long time. 

References

Jim Gray, 'Why do computers stop and what can be done about it?', Symposium on Reliability in Distributed Software and Database Systems, 1986.

By: S.G. Ganesh

The author works in Siemens (Corporate Technology), Bengaluru. His latest book is the best-seller *Cracking the C, C++ and Java Interview*, published by Tata McGraw-Hill. You can reach him at sgganesh@gmail.com.